

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## What You Just Built

You just created a COMPLETE DevOps workflow! Your website is LIVE at:  
<http://54.160.141.125:8080> - Anyone in the world can visit it!

Think of DevOps like a RESTAURANT:

- Cooks (Developers) create the food (write code)
- Waiters (Operations) serve customers (run servers)
- DevOps brings them together to serve food faster!

You did:

1. Wrote website code (like creating a recipe)
2. Packaged it in Docker (like a takeout container)
3. Saved on GitHub (like a recipe book)
4. Deployed to AWS (like opening a restaurant branch)
5. Made it public (like serving customers)

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 1: Create Project Folder

```
mkdir devops-complete-workflow
```

**WHAT:** Creates a new empty folder

**WHY:** You need a place to organize your project files

**HOW:** mkdir = make directory (directory means folder)

**ANALOGY:** *Like creating a folder on your desktop for schoolwork*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 2: Enter the Folder

```
cd devops-complete-workflow
```

- WHAT:** Moves you inside that folder
- WHY:** All your work will now happen inside this folder
- HOW:** cd = change directory (changes your location)
- ANALOGY:** *Like opening a folder to work inside it*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 3: Create Website File

```
nano index.html
```

- WHAT:** Creates and edits your website HTML file
- WHY:** This is the webpage people will see
- HOW:** nano is a text editor (like Notepad in terminal)
- ANALOGY:** *Like writing a story in Microsoft Word*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 4: Create Docker Instructions

### `nano Dockerfile`

- WHAT:** Instructions for Docker to package your app
- WHY:** Tells Docker to use Nginx and copy your HTML
- HOW:** Dockerfile has commands like FROM, COPY
- ANALOGY:** *A recipe: Take bread, add peanut butter, add jelly*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 5: Initialize Git

```
git init
```

- WHAT:** Turns your folder into a Git repository
- WHY:** Git will track all changes to your files
- HOW:** Creates hidden .git folder to store history
- ANALOGY:** *Turning on a camera that records everything*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 6: Stage Files

```
git add .
```

**WHAT:** Marks files to be saved in next snapshot

**WHY:** Git needs to know WHICH files to save

**HOW:** The dot (.) means all files in folder

**ANALOGY:** *Selecting which photos to add to album*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 7: Commit (Save Snapshot)

```
git commit -m 'Initial commit'
```

**WHAT:** Saves a snapshot with a message

**WHY:** Creates a save point you can return to

**HOW:** -m adds a message describing the change

**ANALOGY:** *Taking a photo and writing 'Day 1' on back*



# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 8: Connect to GitHub

```
git remote add origin <URL>
```

**WHAT:** Links your folder to GitHub repository

**WHY:** Tells Git where to upload code

**HOW:** origin is nickname for GitHub URL

**ANALOGY:** *Saving a contact as 'Mom' instead of number*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 9: Rename Branch

```
git branch -M main
```

**WHAT:** Renames branch from master to main

**WHY:** GitHub uses main as default name

**HOW:** -M means move/rename

**ANALOGY:** *Renaming a file to match expectations*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 10: Push to GitHub

```
git push -u origin main
```

- WHAT:** Uploads code from laptop to GitHub
- WHY:** Backs up code online, accessible anywhere
- HOW:** Sends commits to GitHub servers
- ANALOGY:** *Uploading photos to Google Photos*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 11: Launch EC2 Instance

**AWS Console > Launch Instance**

**WHAT:** Creates a virtual computer in Amazon cloud

**WHY:** Need a server that runs 24/7 with public IP

**HOW:** Selected Ubuntu, t2.micro, security settings

**ANALOGY:** *Renting a computer in data center*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 12: SSH into EC2

```
ssh -i nginxp.pem ubuntu@54.160.141.125
```

**WHAT:** Remote login to your EC2 server

**WHY:** Need to access server to run commands

**HOW:** Uses SSH key for secure connection

**ANALOGY:** *Using TeamViewer to control remote PC*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 13: Install Git on EC2

```
sudo apt install git -y
```

- WHAT:** Installs Git software on EC2
- WHY:** EC2 needs Git to download from GitHub
- HOW:** apt is package manager, -y auto-confirms
- ANALOGY:** *Installing Chrome on new computer*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 14: Clone from GitHub

```
git clone <GitHub-URL>
```

**WHAT:** Downloads your code to EC2

**WHY:** EC2 needs the files to run your app

**HOW:** Git copies entire repository

**ANALOGY:** *Downloading from Google Drive*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 15: Install Docker

```
sudo apt install docker.io -y
```

**WHAT:** Installs Docker on EC2

**WHY:** Docker runs containers (isolated apps)

**HOW:** Downloads Docker package

**ANALOGY:** *Installing VirtualBox for VMs*



# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 16: Build Docker Image

```
docker build -t my-nginx-app .
```

- WHAT:** Creates image from Dockerfile
- WHY:** Image is template to create containers
- HOW:** Reads Dockerfile, gets Nginx, copies HTML
- ANALOGY:** *Preparing ingredients (not cooking yet)*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 17: Run Docker Container

```
docker run -d -p 8080:80 --name my-app my-nginx-app
```

**WHAT:** Starts container - APP IS RUNNING!

**WHY:** Container is the running instance

**HOW:** -d=background, -p=port map, --name=label

**ANALOGY:** *Actually cooking the prepared meal*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## STEP 18: Open Port 8080

EC2 > Security Group > Add Rule

- WHAT:** Opens firewall to allow port 8080
- WHY:** AWS blocks all traffic by default
- HOW:** Add Custom TCP, Port 8080, Source 0.0.0.0/0
- ANALOGY:** *Unlocking front door for visitors*

# Complete DevOps Workflow

*Explained Simply for Everyone!*

## Understanding: Localhost vs Public IP

LOCALHOST (127.0.0.1):

- Only YOU on your computer can access
- Like talking to yourself in mirror
- curl localhost:8080 = talking to EC2 itself

PUBLIC IP (54.160.141.125):

- ANYONE on internet can access
- Like your home address
- Anyone can visit your website

PORTS - Think of Apartment Building:

- Public IP = Building address
- Ports = Apartment numbers
- Port 8080 = Apartment #8080

Common Ports:

- Port 80 = HTTP (google.com)
- Port 443 = HTTPS (https://google.com)
- Port 8080 = Development HTTP